

CrashNBurn Firmware Project Guide

Understand the RTOS • Design the software • Configure and build

UB SEDS training

15 September 2026 — proposed training baseline

Contents

1. The RTOS features we use—and why	3
1.1 Three application threads	3
1.2 Queues: give both consumers their own copy	3
1.3 Mutexes: share the physical SPI bus	3
1.4 Scheduling, waits and bounded memory	4
2. A basic overview of the drivers	4
2.1 What the schematic actually supports	5
2.2 Required datasheet reading and citations	5
3. Bring up the SDD template in Overleaf	6
4. Define the requirements	7
4.1 Proposed acquisition and processing baseline	7
5. Build the SDD using RFBoard as an example	8
5.1 Map the template to CrashNBurn	8
5.2 Specify queue and bus contracts	9
5.3 Define durable logs and realistic capacity	9
5.3.1 Address the actual external flash	10
5.3.2 Implement and verify the SPI command layer	10
5.3.3 Follow one record from queue to durable bytes	11
5.3.4 Follow a USB request back out of flash	12
5.4 Define USB download and deletion	12
5.5 Write the diagrams and tests	13
5.6 LaTeX skills you need	14
6. Create the firmware project in CubeMX	15
6.1 Install packages and select the MCU	15
6.2 Apply the schematic pin map	15
6.3 SYS, RCC and clock tree	16
6.4 GPIO and SPI1	17
6.5 USB device, ThreadX and USBX	18
6.6 Generate C and CMake	22
7. Set up VS Code, check dependencies and build	23
7.1 Open the generated project	23
7.2 Install the STM32 extensions	23
7.3 Use Git integration in VS Code	25
7.4 Dependency checklist: install and verify	27
7.5 Use build.py	29
7.6 Make ST code navigation follow the ARM build	30

7.7 Flash, debug and check the board 30

1. The RTOS features we use—and why

CrashNBurn is a small sensor logger built around an **STM32G491CET6**. It reads a BMI088 accelerometer/gyroscope and BMP390 barometer, processes their measurements, and stores raw and processed records in external SPI flash. A computer connects over USB to download and delete old logs.

We use **ThreadX**, a real-time operating system (RTOS), and the **STM32 Hardware Abstraction Layer (HAL)**. ThreadX schedules independent activities and provides safe ways for them to exchange data. HAL supplies the MCU peripheral interface. **USBX** supplies the USB device protocol stack. There is no SEDSNet dependency.

The project is an exercise in building firmware from a reviewed design. This guide supplies requirements, proposed starting settings and examples; it does not claim that the firmware or its hardware tests are complete. Numeric rates, memory sizes and protocol choices are proposals to validate. The accompanying RFBoard SDD illustrates how to document a design; its radio/network architecture is not part of CrashNBurn.

1.1 Three application threads

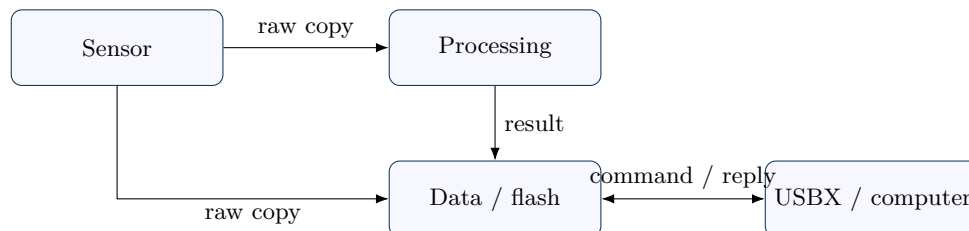
A *thread* (also called a task here) is a function with its own stack that can wait while other threads run. There are three application workers:

Worker	Responsibility	Why it is separate
Sensor	Initialize and read all populated sensors; publish samples.	Sensor acquisition needs regular service.
Processing	Convert/compensate measurements and calculate defined outputs.	Calculation should not delay the next acquisition or own flash.
Data	Read, program and erase SPI flash; manage logs and storage commands.	Slow storage needs one owner and explicit recovery rules.

ThreadX and USBX may also create timer and USB service threads. Document these as infrastructure; “three tasks” means three **application workers**, not exactly three scheduler objects in the entire system.

1.2 Queues: give both consumers their own copy

A queue transfers messages between threads. The sensor must send each sample to **two queues**: one for processing and one for storage. Processing then sends its result to storage. A single queue with two readers does not broadcast; each message normally goes to only one reader.



Give raw and processed records a session ID, input sequence and timestamp. Storage can then correlate them even when they arrive in a different order. Independently attempt both sensor sends; failure on one path must not skip the other. Count dropped messages per path.

1.3 Mutexes: share the physical SPI bus

All sensors and the flash share **SPI1**. A mutex is a lock that lets only one thread perform a bus transaction at a time. Create one SPI mutex with **priority inheritance (TX_INHERIT)**: when the high-priority sensor waits on a lock held by data, ThreadX can temporarily raise the lock owner’s priority so it can finish and release the bus.

The transaction sequence is: acquire mutex, assert exactly one chip select, transfer the complete command/address/data sequence, deassert chip select, release mutex. On error, still deselect and unlock. Do

not hold the mutex while waiting for sensor conversion, flash internal program/erase completion, queues or USB transmission.

A flash erase can take much longer than the SPI command that starts it. Release SPI after the command and poll flash status later. Other **slaves** can use SPI during the flash's internal busy interval; the same flash may still be unable to serve reads.

1.4 Scheduling, waits and bounded memory

Feature	Use in this project
Priorities	Sensor first, processing next, data last; smaller ThreadX number means higher priority.
Tick / timed waits	Release acquisition periodically and give operations finite deadlines.
Blocking queue receive	Let processing sleep until a sample is available.
Event flags / short timed wait	Wake data for records, USB commands or a flash-poll deadline.
Static queues and stacks	Make the RAM budget predictable and avoid runtime allocation surprises.
Error/status checks	Detect failed thread creation, queue overflow and exhausted middleware pools.

Proposed starting priorities/stacks: sensor **5 / 2048 bytes**, processing **10 / 4096 bytes**, data **15 / 4096 bytes**. Use preemption threshold equal to priority and no time slice initially. These stacks are initial allocations; measure their high-water marks and include middleware/system stacks in the budget.

Use a **1 kHz ThreadX tick** as the starting point, with an absolute 10 ms acquisition schedule. Incrementing the next deadline avoids the drift of “work, then sleep 10 ms.” If a deadline is missed, record it and resume a defined schedule instead of spinning through an unbounded catch-up loop. The HAL timebase will use TIM6; ThreadX owns its port tick and exception handlers.

For fixed messages on this Cortex-M4 port, 64 bytes fit 16 32-bit **ULONG** words. ThreadX queue message size is in **words**, but queue backing storage size is in **bytes**. Verify `sizeof(ULONG) == 4`, use aligned storage and assert envelope size. Do not enqueue pointers to a stack-local sample. [ThreadX queue implementation](#).

2. A basic overview of the drivers

A driver turns device-specific register transactions into an interface the application can use, such as “initialize the accelerometer” or “read one flash page.” CubeMX initializes **MCU peripherals**; it does not generate the complete BMI088, BMP390, flash logger or application algorithms.

Layer	What you write or configure
HAL / generated platform	MCU clocks, GPIO, SPI1, USB peripheral, interrupts and timebase.
Shared SPI helper	Mutex, correct chip-select selection, bounded HAL transfers and cleanup.
BMI088 driver	Separate accel/gyro identities, resets, power/range/rate configuration and register reads.
BMP390 driver	Reset, identity/status, calibration coefficients, raw pressure/temperature and supported rate configuration.
W25Q32 driver	Identity, status, read, write-enable, page program, sector erase, busy deadlines and bounds.
Log store	Session allocation, record serialization, commit/CRC, recovery and deletion.

Layer	What you write or configure
USB command adapter	Framing, parsing, queueing commands and returning copied replies.

The sensor thread owns sensor initialization and acquisition. Pass immutable calibration and scale information to processing before acquisition starts. Processing performs physical conversion and compensation. The data thread is the sole runtime flash owner; a USB callback must enqueue a request rather than directly reading or erasing flash.

BMI088 accel and gyro have separate chip selects and device-specific read behavior. Check SPI activation and dummy bytes. BMP390 requires its calibration coefficients for compensation. Start with identity checks before implementing processing. Consult the [BMI088 datasheet](#), [BMP390 datasheet](#) and [Winbond documentation for W25Q32JV](#).

2.1 What the schematic actually supports

Schematics.pdf is the hardware source: sheet 2 contains the MCU and connector wiring; sheets 3–5 contain the sensors and flash. The flash is **W25Q32JVSSIQ, 32 Mbit / 4 MiB**. The sensor interrupt pins are not connected to the MCU, so this baseline polls status instead of inventing EXTI connections.

The G491 has **USB device** capability. Connect it to a **computer acting as host**. A thumb drive is also a device; using one directly requires added USB host hardware and a revised design. The USB-C connector alone does not make the board a host. [STM32G491 datasheet](#).

Before bring-up, resolve these schematic-specific checks in the SDD:

- Sheet 2 labels PA13/pin 37 as JTDI/NC and PA15/pin 39 as JTMS/SWDIO. The MCU's actual SWDIO is **PA13/pin 37**. Check continuity and connect the probe to the real signal; software cannot move SWDIO to PA15. Confirm probe reference voltage, ground and reset connections too.
- The regulator is supplied from the battery path. USB VBUS is not shown feeding it. Supply the board as intended; do not assume USB powers the MCU. VBUS sensing is also not shown at an MCU pin: review self-powered attach/detach behavior and any required hardware change.
- Flash IO2 and IO3 are unconnected. Verify the exact fitted suffix and QE/status-register behavior. Do not copy blanket status writes from a different flash variant.
- Confirm the fitted 8 MHz HSE crystal and optional 32.768 kHz LSE against the BOM.

2.2 Required datasheet reading and citations

The repository includes the **complete manufacturer PDFs in datasheets/**. Keep these beside this guide when working offline. Start each driver by reading its device's interface, startup, register and electrical sections. CubeMX peripheral settings do not replace those instructions.

Device / local PDF	Included revision	Read before implementing
W25Q32JV / W25Q32JV-SPI-Flash.pdf	Rev K, April 2026	Geometry; status/protection and IQ suffix; command/address timing; page program and erase limits. Detailed viewer-page references appear in §5.3.2.
BMP390 / BMP390-Barometer.pdf	1.7, March 2021	Power/measurement modes; register map and calibration; §5.3 SPI read/write and dummy bytes; compensation appendix.
BMI088 / BMI088-IMU.pdf	1.9, January 2024	§3 initialization; §4 power, data and rate settings; §5 separate register maps; §6.1 separate accel/gyro SPI formats.

Device / local PDF	Included revision	Read before implementing
STM32G491 / STM32G491-MCU.pdf	DS13122 Rev 4, April 2024	Exact CET6/LQFP48 pinout and alternate functions; clocks/power; SPI/USB electrical requirements and memory limits.

Local filenames above are relative to `datasheets/`. Its README records original manufacturer download links, document IDs and SHA-256 hashes. Use the PDF's internal revision, since a manufacturer's download filename may not match the printed document ID. The STM32 family PDF covers several packages: always select the **STM32G491CET6 LQFP48** column and pin diagram.

In the SDD reference section, assign IDs such as **HW-FLASH**, **HW-BARO**, **HW-IMU** and **HW-MCU**, and record title, manufacturer, document ID/revision, URL, access date and local filename. Cite the relevant section/page beside every register sequence, timing limit, pin assignment and capacity assumption. For example: "A program transaction must stay within one 256-byte page [HW-FLASH, Rev K, printed p.35 / PDF p.36]." Separate these manufacturer limits from your proposed scheduling margins and measured results.

For MCU register programming, also consult ST's applicable reference manual and errata; the MCU datasheet mainly establishes device capabilities, pin assignments and electrical limits. HAL function behavior comes from the installed CubeG4 HAL documentation/source. Record those package/manual versions in the SDD as well. Review references whenever hardware or middleware changes.

3. Bring up the SDD template in Overleaf

Do this before starting the firmware implementation. An SDD records requirements, ownership, interfaces, timing, failure handling and verification. Overleaf edits the LaTeX source and compiles a PDF; **XeLaTeX** is the compiler this template uses.

1. Sign in to Overleaf. On the projects dashboard choose **New Project** → **Upload Project** and upload `UBSEDS_SDD_Template_Overleaf.zip`. Keep the entire folder structure intact. [Overleaf upload instructions](#).
2. Rename it **CrashNBurn Software Design Document**.
3. Open project **Settings/Menu**, set **Main document** = `main.tex`, and **Compiler** = **XeLaTeX**. Record the TeX Live version. The menu's location varies by editor version. [Compiler settings](#).
4. Click **Recompile** before making changes. Get the untouched template compiling first.
5. Open `metadata.tex` and replace its existing project/document fields. Keep guidance visible while writing.
6. Open `RFBoard26_Software_Design_Document.pdf` beside your editor. Optionally upload the RFBoard ZIP as a **separate** Overleaf project to inspect its LaTeX. Do not merge its files into your CrashNBurn template.

Template file	Purpose
<code>metadata.tex</code>	Cover information, revision, status, baseline and guidance switch.
<code>main.tex</code>	Packages and section inclusion order.
<code>ubseds-brand.sty</code>	Shared UB SEDS branding.
<code>sections/</code>	Requirements, architecture, design, tests and references.
<code>figures/</code>	Editable TikZ context, startup, sequence and state diagrams.
<code>assets/</code>	Logo and any added screenshots, with their provenance.

Example replacements in `metadata.tex`:

```
\newcommand{\ProjectName}{CrashNBurn}
\newcommand{\DocumentID}{UBSEDS-SDD-CBN-001}
\newcommand{\DocumentVersion}{0.1}
\newcommand{\DocumentStatus}{DRAFT --- PROPOSED DESIGN}
\newcommand{\DocumentAuthor}{Your name / firmware team}
```

```
\newcommand{\DocumentReviewer}{Assigned reviewer}  
\newcommand{\SourceBaseline}{Repository; commit; local changes}  
\newcommand{\TargetPlatform}{STM32G491CET6; board revision TBD;  
  ThreadX and STM32 HAL}
```

Replace the existing definitions rather than adding duplicate `\newcommand` lines. Preserve the template's other fields, `\DocumentDate`, `\newif\ifshowguidance` and `\showguidancetrue`.

Checkpoint: your project compiles with the CrashNBurn title and a clearly marked draft status. The original template ZIP and the RFBBoard reference remain unchanged.

4. Define the requirements

A requirement states an observable behavior. A design decision chooses how to achieve it. A test result records what a particular firmware/board version actually did. Keep those three things distinct; “proposed” and “not run” are useful, honest statuses.

Copy the following IDs into `sections/01-purpose-and-requirements.tex`, then add an owner and approval status. Keep IDs stable as wording evolves.

ID	Required behavior	Verification
CBN-001	Use HAL and ThreadX, build with CMake/GCC, and have no SEDSNet dependency.	Build/map review.
CBN-002	One sensor task shall read all populated sensors: acceleration, angular rate, pressure and sensor temperature.	Identity/stimulus tests.
CBN-003	Independently deliver each sample to processing and data queues with matching sequence/time identity; count each failed delivery.	Fan-out/overflow tests.
CBN-004	Processing shall perform specified compensation/conversion and send results to data with the input sequence reference.	Known-input tests.
CBN-005	Data shall exclusively own runtime flash reads, writes, erases and metadata.	Ownership review.
CBN-006	Persist raw and processed records in external SPI flash with format version, type, timestamp, length and corruption detection.	Format/readback tests.
CBN-007	Serialize SPI access with bounded transactions and exactly one asserted chip select.	Logic analyzer/load test.
CBN-008	A computer over USB shall list and download retained logs with integrity information and resumable reads.	Host export/disconnect test.
CBN-009	Explicitly delete selected old logs; never erase on download; reject deletion of active or pinned logs.	Erase protection tests.
CBN-010	Reset/power loss shall preserve unrelated committed records and reject incomplete records; interrupted erase shall be recoverable.	Power-cut tests.
CBN-011	Queue exhaustion, full flash, device faults and USB disconnect shall have bounded handling and visible status/counters.	Fault injection.
CBN-012	Maintain generated settings, build instructions, resource budgets, manufacturer references with revision/section citations, and test evidence in the SDD.	Review/regeneration/reference audit.

4.1 Proposed acquisition and processing baseline

Start with a **100 Hz sensor-task release**, BMI088 accel/gyro ODR 100 Hz, and BMP390 pressure/temperature target 25 Hz. Choose actual supported bandwidth and oversampling settings. Poll pressure status and record freshness/age so a 25 Hz value is not presented as 100 Hz fresh measurements.

Minimum proposed processing: acceleration in m/s², angular rate in rad/s, compensated pressure in Pa, sensor temperature in °C, and barometric altitude relative to a recorded reference pressure. Specify mounting axes/signs, ranges, scale factors, calibration, the altitude formula, reference capture and invalid-input handling. Sensor die temperature is not necessarily ambient temperature.

Use a monotonic timestamp and session ID. If time comes from a 1 kHz tick, its resolution is 1 ms even if encoded in microseconds. Handle tick wrap. No GPS/UTC time source is implied by this board.

A Kalman filter is optional additional scope. If selected, specify its state, model, noise parameters, units, initialization, reset behavior and test vectors. The schematic note does not by itself define an algorithm requirement.

Proposed nominal targets: no missed 10 ms releases during a 60 s test with non-full flash; acquisition start jitter at most 1 ms; processing within 10 ms of queue delivery; no unexplained nominal loss. Measure these before accepting them. Counted overflow is an error-handling behavior, not proof of lossless operation.

5. Build the SDD using RFBoard as an example

Write enough detail that another teammate can implement or review the firmware without this conversation. Work through the template, recompile after small changes, and tie every requirement to a design location and a test.

5.1 Map the template to CrashNBurn

Template section file	Write this content
00-document-control.tex	Revision history, reviewers, status and source baseline.
01-purpose-and-requirements.tex	CBN requirements, scope, targets and exclusions.
02-context-and-architecture.tex	Three workers, computer host, USB adapter, sensors and flash.
03-drivers-and-platform.tex	Pin map, clocks, HAL/ThreadX/USBX and device initialization.
04-tasks-and-concurrency.tex	Owners, priorities, stacks, queues, mutex and overflow rules.
05-control-flow.tex	Startup, acquisition, processing, stop handshake and maintenance states.
06-network-and-interfaces.tex	Retitle to USB and internal interfaces; define command contracts; no SEDSNet.
07-data-and-persistence.tex	Record bytes, CRC, flash map, sessions, commit, recovery and erase.
08-update-and-recovery.tex	SWD programming and boot recovery; OTA is out of scope.
09-resources-and-timing.tex	RAM/map, flash duration, queue capacity and timing evidence.
10-faults-and-security.tex	Device faults, malformed USB, erase confirmation and recovery.
11-build-and-deployment.tex	CubeMX settings, VS Code, dependencies and build/flash commands.
12-verification.tex	Tests, expected tolerances, evidence and actual status.
13-decisions-and-open-work.tex	Open decisions, owners and closure criteria.
14-references.tex	Schematic sheets, datasheet revisions, code commits and tool docs.
15-data-dictionary.tex	Field types, offsets, units, validity and producers/consumers.

RFBoard uses numbered files `sections/01.tex` through `19.tex`. Read its headings rather than assuming its numbers match the template. Use `04.tex` for the style of task/ownership documentation, `07.tex` for persistent versus volatile data, `09.tex` for budgets, `11.tex` for deployment, and `16.tex/17.tex` for interface and qualification detail.

Copy the **documentation habits**: define owners, limits, failure paths and evidence. Replace RFBoard's CAN-FD, radio, GPS/network time, SEDSNet and LaunchCore architecture with CrashNBurn's actual design. Do not reuse

RFBoard's test counts, memory figures or claimed results.

5.2 Specify queue and bus contracts

Proposed fixed 64-byte envelopes:

Queue	Producers → consumer	Depth / backing bytes
<code>sensor_to_process</code>	Sensor → processing	32 / 2048
<code>records_to_data</code>	Sensor and processing → data	128 / 8192
<code>usb_commands</code>	USB adapter → data	8 / 512
<code>usb_responses</code>	Data → USB adapter	8 / 512

These arrays consume **11,264 bytes**, excluding queue objects and any separate USB chunk pool. Use initialized, aligned `ULONG` envelopes and explicit encoding. Oversized data must be split or use a separately designed owned-buffer pool; never silently truncate it.

```
/* Illustration: encode the complete initialized envelope first. */
ULONG message[16] = {0};
encode_sensor_envelope(message, &sample);
if (tx_queue_send(&sensor_to_process, message, TX_NO_WAIT)
    != TX_SUCCESS) {
    process_delivery_drops++;
}
/* This attempt must happen even if the first send failed. */
if (tx_queue_send(&records_to_data, message, TX_NO_WAIT)
    != TX_SUCCESS) {
    raw_log_drops++;
}
```

Processing independently counts failed result sends. Proposed overflow policy: drop the **new** message only on the full path, maintain counters and expose sequence gaps. Do not let slow storage block acquisition indefinitely.

Data must service all inputs fairly: for example, drain up to eight records, advance a pending flash operation, then service one USB command step. Use an event or short timed wait; do not sleep indefinitely on an empty record queue while a command waits elsewhere. All SPI transfers and busy polling need finite, datasheet-based deadlines.

At 3 MHz, a 260-byte SPI transaction takes about 0.69 ms on the wire before software overhead. Measure its effect on acquisition jitter. Never hold the SPI mutex across a flash erase's internal busy interval.

5.3 Define durable logs and realistic capacity

Use a raw log store initially; a filesystem is not required. Confirm the exact part's 256-byte program pages and 4 KiB erase sectors against its approved datasheet. Proposed allocation: two sectors for alternating metadata journals; sector-aligned sessions in the remaining flash. Record actual address bounds.

Define on-flash bytes independently from compiler C structure layout. One proposed 80-byte slot is:

Offset	Bytes	Field
0	4	Magic
4	1	Format version
5	1	Record type
6	2	Payload length
8	4	Session ID
12	4	Input sequence
16	8	Monotonic timestamp
24	48	Payload, unused bytes zeroed
72	4	CRC

Offset	Bytes	Field
76	4	Commit word

Specify little-endian integer encoding and any floating-point representation. Proposed CRC: CRC-32/ISO-HDLC, reflected polynomial 0xEDB88320, initial/final XOR 0xFFFFFFFF, covering bytes 0–71. Commit is 0x00000000; erased 0xFFFFFFFF is uncommitted.

Three slots fit a 256-byte page with 16 bytes padding. Never cross a page boundary in one program command. Program/verify body and CRC before programming the commit word, after confirming the chip permits the required partial-page program sequence. Otherwise design a separate commit-record scheme. Check write-enable, status and bounded completion for every modification.

On boot, choose the newest valid metadata journal by generation/CRC/commit, scan session allocations and accept only valid committed slots. Quarantine torn slots; do not reuse them without sector erase. Replace and verify metadata before reclaiming the last valid copy. Journal session boundaries and deletion, not every sample. Do not reuse session IDs after reset.

At **100 raw + 100 processed records/s**, 80-byte records use 16,000 bytes/s; page padding raises occupied flash to approximately 17,067 bytes/s. The ideal capacity is roughly **245 seconds (4.1 minutes)** before additional headers/reserved space. Decide the required recording duration before accepting rates. This is not hours of logging.

A 128-record queue holds only **0.64 seconds** at 200 records/s. Compare against worst-case program/erase stalls and RAM limits. Initial full policy: preserve old logs, report FULL, count rejected persistent records and keep sensor/processing responsive. Pre-erase free space in maintenance mode where possible. Do not silently overwrite retained logs.

Delete only closed, unpinned, sector-aligned sessions. First durably journal the target session/sector set as deleting; erase and blank-check sectors; then journal completion/free space. Reset recovery must resume or report the operation without touching unrelated sessions. Report completion after verified erasure, not on receipt of the command.

5.3.1 Address the actual external flash

32 Mbit means 4,194,304 bytes (4 MiB), not 31 MB or 32 MB. There are 16,384 pages of 256 bytes and 1,024 sectors of 4,096 bytes. The address range is 0x000000 through 0x3FFFFFF, inclusive. 0x400000 is the exclusive upper bound. If the fitted part changes, revise identification, capacity and timing together before accepting it.

Region	Inclusive byte addresses	Proposed use
Sector 0	0x000000–0x000FFF	Metadata journal A
Sector 1	0x001000–0x001FFF	Metadata journal B
Sectors 2–1023	0x002000–0x3FFFFFF	Session headers and log slots

This leaves 1,022 sectors for sessions. Reserve a header page at each session's beginning and record its allocated sectors durably. A queued record is still only in **RAM** until data has programmed and verified its committed flash slot. Queue success is not a persistence acknowledgment.

The flash is attached to SPI1, with **PB14 as its active-low chip select**. It is not a memory-mapped C array in this design. `HAL_FLASH_Program`, linker addresses and the `build.py` firmware programming address apply to the STM32's internal flash; they do not write this external log chip. The external driver uses bounded HAL SPI transfers, such as `HAL_SPI_Transmit` and `HAL_SPI_TransmitReceive`, underneath the shared SPI mutex.

5.3.2 Implement and verify the SPI command layer

Open the included **W25Q32JV-SPI-Flash.pdf** while implementing this layer. The following page numbers are **PDF viewer pages**, which are one greater than the printed Winbond page numbers. Read pages 22–23 for identification/commands, 25–28 for enable/status/read, 36 for programming, 38 for sector erase and 66 for maximum operation times.

Operation	Bytes sent on MOSI	Result / rule
Identify	9F, then clock 3 bytes	MISO should return EF 40 16 for the specified IQ part.
Status 1	05, then clock 1 byte	Bit 0 BUSY; bit 1 WEL (write-enable latch).
Write enable	06	Its own CS transaction before each program/erase.
Read	03 A23..16 A15..8 A7..0, then clocks	MISO supplies consecutive data; no extra dummy phase for 03.
Page program	02, 3 address bytes, 1–256 data bytes	Never cross a 256-byte page boundary.
Sector erase	20, 3 address bytes	Use a validated, 4 KiB-aligned sector base.

Clock each byte most-significant bit first. Transmit the **24-bit address high byte first**, even though record fields are serialized little-endian. During reads, transmit expendable bytes (for example FF) to generate clocks and collect MISO. Keep CS low across command, address and data phases, including separate HAL calls; keep the mutex for that complete transaction. Return CS high and release the mutex on every exit path.

At initialization, meet the power-up timing, read the ID and inspect protection/status. Reject an unexpected part rather than guessing its geometry. Rev K PDF page 17 states the IQ suffix has QE fixed high by default and still supports standard SPI; IO2/IO3 are not used by these standard commands. Verify the fitted suffix against the schematic and do not blindly clear QE or write status/OTP lock bits.

For every range, reject `address >= capacity` and `length > capacity - address` before transferring; this avoids integer-overflow mistakes. Reject zero-length program operations. Split an arbitrary write with `n = min(remaining, 256 - (address & 255))`. The chip wraps excess page-program bytes within the same page, so a driver that simply streams across a page can corrupt earlier data.

Erase makes bytes FF. Programming changes bits from 1 to 0; erase is required to restore 0 to 1. Allocate only verified erased space. A closed retained log must never be erased just to satisfy a new write.

5.3.3 Follow one record from queue to durable bytes

Implement this as a data-thread state machine, with bounded steps:

1. Receive a raw or processed envelope from `records_to_data`. Validate its type/length and encode the 80-byte format above; do not dump compiler structures directly.
2. Allocate the next erased slot in the active session. For illustration, a session starting at 0x002000 reserves that page for its header, so its first record starts at 0x002100.
3. Check BUSY is clear; issue 06 in its own transaction, then read status and verify WEL. Issue 02 00 21 00 followed by the **76 bytes** of body and CRC. Raise CS to start programming and release SPI.
4. Poll status in later bounded steps while allowing sensor SPI transactions. Once ready, read the 76 bytes back and compare/validate CRC. A clear BUSY bit alone does not prove successful programming; protection or a rejected command could leave data unchanged.
5. For the proposed separate commit-word scheme, issue a fresh write-enable and program four zero bytes at 0x00214C; wait and read back the complete slot. Confirm the selected part permits the partial-page sequence before accepting this format. Each programmed byte starts erased and is written once.
6. Only then count the record as durable and advance the committed-log length. The next slots start at 0x002150 and 0x0021A0. Skip the page's final 16 padding bytes; the next record page begins at 0x002200.

Program failure, readback mismatch or uncertain completion must produce a fault/quarantined slot, not an unconditional retry over partly programmed bytes. Boot recovery accepts only slots with the expected commit marker, valid format and CRC. An absent raw or processed counterpart is represented by sequence gaps and counters; it is not silently fabricated.

The datasheet's maximum page-program time is **3 ms** and 4 KiB sector-erase time is **400 ms** (PDF page 66). Initial software deadlines of **10 ms per program** and **500 ms per sector erase** include a proposed scheduling

margin; validate them over supported conditions. Two program operations per slot mean worst-case throughput can be much worse than the typical rate. The 100 + 100 records/s baseline therefore requires measurement and may require batching or lower rates. Keep flash internal-busy waiting outside the SPI mutex and do not claim that a RAM queue can absorb sustained overload.

For deletion, durably record deletion intent, validate each target sector belongs exclusively to that closed/unpinned session, write-enable, issue 20 plus sector base, poll completion, and blank-check in bounded chunks. Finish the metadata update only after successful verification. Avoid chip erase in the normal user protocol because it would remove unrelated sessions and metadata.

5.3.4 Follow a USB request back out of flash

The protocol in the next section describes the host-facing commands. Its implementation path is:

1. The USB adapter parses **READ id offset count** and enqueues a command. The **data thread** validates the session, export lease, committed logical length and requested count; it alone accesses flash.
2. Translate the logical export offset to a valid record slot. Logical export is the concatenation of committed 80-byte slots, in physical append order, excluding session headers, torn slots and page padding. Implement an index or bounded scan to perform this mapping. Do not assume **flash_address = session_base + offset**.
3. Read a physical slot with 03 and its three address bytes. For the example above, send 03 00 21 00, then generate clocks to receive 80 bytes. Check commit/CRC before copying the requested portion into a response buffer. Split a chunk across slots as needed.
4. Release the SPI mutex before returning the copied chunk to the USB adapter. Attach the chunk length and CRC; the adapter transmits the framed reply. Use an owned response buffer with an explicit release rule, not a pointer to a vanished stack variable.
5. The host verifies the CRC, saves the binary bytes and advances the logical offset. On reconnect, check session ID/generation before resuming. Only an explicit separate deletion command can erase the saved session.

Do not read this flash while it is internally busy programming/erasing. Queue, defer or return a bounded BUSY response; never block the sensor task. A slow/disconnected computer must not retain the SPI mutex or stall log storage indefinitely.

Bring-up exercise: read the JEDEC ID, use a deliberately allocated scratch sector in maintenance mode, erase/blank-check it, write a known pattern spanning two pages using split commands, and verify byte-for-byte readback. Then log raw and processed records, stop/close the session, download it over USB and compare its decoded sequence numbers and CRCs. Test the last legal byte, rejected out-of-range operations, page padding, torn writes and interrupted deletion. Include logic-analyzer captures of CS/SCK/MOSI/MISO and reference the command diagrams in the SDD.

5.4 Define USB download and deletion

Use USBX CDC ACM as a byte stream between board and computer. USB packet boundaries are not command boundaries. Proposed requests are LF-terminated ASCII lines, optionally preceded by CR, at most 128 bytes, with one outstanding request per connection. Discard malformed/overlength input through LF; never execute a partial command.

Command	Contract
STATUS	Versions, state, IDs, free space and fault/drop counters.
LIST	Closed sessions, generation, byte length and integrity metadata.
STOP	Establish a producer cutoff, drain processing/storage and close the active session.
START	Create a new session when healthy and space exists.
READ id offset count	Read 1–512 bytes from a closed session after bounds checking; no mutation.
ERASE_PREPARE id	Return generation and short-lived confirmation token.
ERASE_CONFIRM id generation token	Delete the matching closed, unpinned session in maintenance mode.

Command	Contract
OP_STATUS operation_id	Return erase progress, completion or failure.

Proposed read response: DATA id offset length crc32\n, then exactly length binary bytes, then LF. The client must honor the byte length even if payload contains LF. Define CRC over the chunk with the same algorithm above. Other replies are OK ... \n or ERR code ... \n. Resume reads by offset only after checking the session identity/generation.

Pin the exported session for the connection's bounded export lease; release on close, disconnect or inactivity timeout. Reject deletion while pinned. Bind a proposed 30-second erase token to the connection, ID and generation, consume it once and invalidate it on disconnect. Duplicate destructive requests must not accidentally execute twice.

For STOP, sensor acknowledges its final logged sequence, processing drains earlier accepted samples and acknowledges its final result, then data drains/commits and closes. Do not close while processing can still enqueue results. Define error/incomplete-session behavior if the handshake times out.

Data copies a bounded read chunk to an available response buffer and releases SPI before USB transmission. The USB adapter must not retain stack-buffer pointers or indefinitely block data waiting for a host. If USBX requires blocking receive calls, use its infrastructure application thread, not an application worker. Include its stack/priority in the SDD.

User workflow: power/connect, STATUS, STOP, LIST, download by offset, verify CRCs, save a local file/manifest, explicitly prepare/confirm deletion of that old session, poll completion, LIST again. Download never implies erase. A host client implementing this protocol is part of the firmware exercise; build.py is not a log-download client.

5.5 Write the diagrams and tests

Update figures/context.tex, startup.tex, sequence.tex and state.tex. Show actual owners and transitions: BOOT → READY → LOGGING → STOPPING → MAINTENANCE, with FULL/FAULT branches, guards and recovery. A sequence diagram should show both independent sensor sends or the download/erase handshake.

Test	Evidence needed
Build/regeneration	Debug/Release build, correct target, no SEDSNet and reviewed regeneration diff.
Sensor/processing	Identity checks, physical stimuli, known compensation/conversion vectors, stale/invalid handling and tick wrap.
Queue paths	Both consumers receive nominal sequences; independently full queues produce exact counters without suppressing the other path.
Bus ownership	No overlapping chip selects; bounded bus locks; only data calls flash operations.
Persistence	Page/sector boundaries, CRC/length/version errors, full capacity and power loss during body, commit, journal rollover and erase.
USB	Byte-exact export, disconnect/resume, malformed commands, wrong/expired tokens and protected sessions.
Load/faults	Measured jitter/stacks/queues; absent sensors, stuck busy, SPI timeout and disconnected host cannot deadlock workers.

For each result record board identity, firmware commit/dirty state, .ioc and tool versions, setup, expected tolerance, actual measurement, pass/fail/not-run, evidence path, tester and date. Tie results to CBN requirement IDs. Do not mark a hardware test passed because a document or empty generated project compiled.

5.6 LaTeX skills you need

Use blank lines between paragraphs, semantic headings and automatic references:

```
\section{Tasks and concurrency}\label{sec:tasks}
The sensor sends each sample to two independent queues.
\textbf{CBN-003:} Both sends must be attempted.
\begin{itemize}
  \item Count each failed send separately.
  \item Preserve the sample sequence in both copies.
\end{itemize}
See Section~\ref{sec:tasks}.
Use \texttt{TX\_NO\_WAIT} for nonblocking sends.
```

Escape literal %, &, _, #, \$ as \%, \&, _, \#, \\$. A bare % begins a comment. Use \url{...} for a URL and \path{App/sensor_task.c} for a path. Inside `verbatim`, type code literally. Do not use repeated \\ for paragraph spacing.

The template provides a wrapping Y column and SDDTable:

```
\begin{SDDTable}[lYY]
ID & Requirement & Verification \\
\midrule
CBN-003 & Deliver to both consumers. & Independent overflow tests. \\
CBN-009 & Protect active logs. & Reject active-session erasure. \\
\end{SDDTable}
```

Use & between cells and \\ to end rows. For multipage tables use `longtable`, already loaded by the template. Explain equations and units, for example queue capacity $N \geq \text{ceil}(\text{rate} * \text{maximum_stall}) + \text{margin}$; measure the maximum stall rather than substituting a typical value.

To add a screenshot, upload it to `assets/`, ensure `\usepackage{graphicx}` is in the preamble, then:

```
\begin{figure}[htbp]
  \centering
  \includegraphics[width=\linewidth]{assets/cubemx-spi1.png}
  \caption{Reviewed CrashNBurn SPI1 settings; CubeMX version X.}
  \label{fig:spi1}
\end{figure}
See Figure~\ref{fig:spi1}.
```

Put labels after captions. Figures may move to a nearby suitable page. Attribute borrowed screenshots and distinguish them from your actual configured project.

The template includes TikZ. Edit existing diagram nodes/edges, or start with:

```
\begin{tikzpicture}[node distance=15mm and 18mm]
  \node[draw] (s) {Sensor};
  \node[draw,right=of s] (p) {Processing};
  \node[draw,below=of p] (d) {Data};
  \draw[-{Latex}] (s) -- node[above]{raw} (p);
  \draw[-{Latex}] (s) |- node[pos=.75,below]{raw} (d);
  \draw[-{Latex}] (p) -- node[right]{result} (d);
\end{tikzpicture}
```

Keep node names unique and define them before drawing edges. Check whether the existing file already contains a `figure` environment before wrapping it in another one.

Compile problem	First fix to try
<code>fontspec</code> compiler error	Select XeLaTeX.
Command already defined	Replace existing metadata instead of duplicating it.
Missing \$	Check unescaped _ and math delimiters.

Compile problem	First fix to try
Extra alignment tab	Count table columns and escape literal <code>&</code> .
File not found	Check upload, filename case and relative path.
Undefined control sequence	Check spelling/package; fix the first error.
Runaway argument	Match braces and begin/end environments.
Reference ??	Check labels and recompile again.
Overfull box	Wrap/shorten wide content and inspect the PDF.

When ready for review, resolve or explicitly track TBDs, replace illustrative content, and set `\showguidancefalse`. That switch hides guidance, not unresolved TBDs. Recompile and inspect the whole PDF. Download **both the PDF and source ZIP**, using matching revision names. [Overleaf project menu](#).

Locally, extract the template or example into its own directory and run from the folder containing `main.tex`:

```
latexmk -xelatex -interaction=nonstopmode -halt-on-error \
  -outdir=build main.tex
```

The result is `build/main.pdf`. This compiles documentation; firmware uses CMake and `build.py`.

6. Create the firmware project in CubeMX

Now translate the reviewed design into peripheral and middleware settings. Keep the `.ioc` with the firmware and record the tool/package versions in the SDD.

Screenshot note: the following are genuine ST CubeMX screenshots, recovered from the previous guide: UM1718 Rev 50 (GPIO, clock tree, CMake) and ST's G474 USBX CDC tutorial (USB, component selector, middleware settings). They are **reference UI examples**, not captures of a completed CrashNBurn configuration. Use the tables in this chapter for actual values. [ST UM1718](#); [ST USBX CDC tutorial](#).

6.1 Install packages and select the MCU

1. Complete the software dependency checklist in section 7.4, including STM32CubeMX and the STM32CubeG4 firmware package.
2. In the embedded software package manager, install **X-CUBE-AZRTOS-G4**. It supplies the G4 ThreadX/USBX integration. [ST package page](#).
3. Choose **New Project** → **MCU/MPU Selector** and select **STM32G491CET6, LQFP48**. Use the MCU, not a Nucleo board preset.
4. Save `crashnburn.ioc` in this repository. Confirm the final generated `CMakeLists.txt` will be beside `build.py`, not in an unintended nested folder.
5. Record the installed versions. The historical sibling project used CubeMX 6.16.1, G4 firmware 1.6.2 and expansion pack 2.0.0, but is incomplete: it omitted flash CS and SWD and enabled unnecessary options. Start fresh with these settings rather than treating it as validated firmware.

6.2 Apply the schematic pin map

Function / net	MCU pin	Physical pin	Assignment
SPI1 SCK / MISO / MOSI	PA5 / PA6 / PA7	13 / 14 / 15	SPI1, AF5
CS_Accel	PB11	25	GPIO output, High
CS_Gyro	PB12	26	GPIO output, High
CS_BMP390	PB13	27	GPIO output, High
CS_Flash	PB14	28	GPIO output, High
USB DM / DP	PA11 / PA12	33 / 34	USB device
LED D2	PA8	30	GPIO output, Low
Buzzer Q1	PC13	2	GPIO output, Low
HSE	PF0 / PF1	5 / 6	Crystal oscillator

Function / net	MCU pin	Physical pin	Assignment
LSE, optional	PC14 / PC15	3 / 4	Leave disabled initially
Actual SWDIO / SWCLK	PA13 / PA14	37 / 38	SYS Serial Wire
Optional debug TX / RX	PA2 / PA3	10 / 11	USART2, AF7

PB13/PB14 must remain chip-select GPIOs, even if CubeMX offers SPI2 functions. PA4 hardware NSS is unused. Sensor interrupts are unconnected; do not enable invented EXTI inputs.

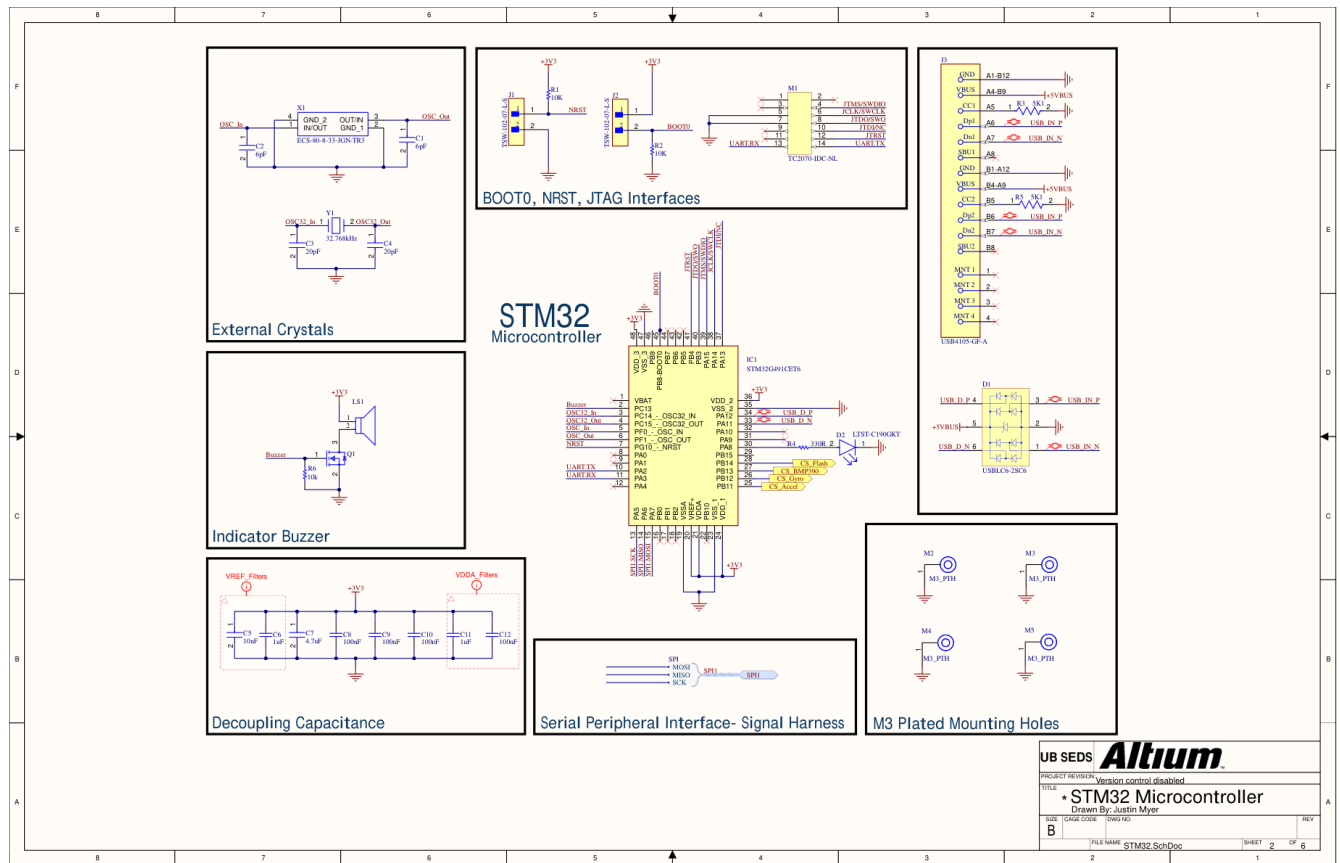


Figure 1: Schematic sheet 2: MCU, USB, clocks and debug wiring. Use the supplied original PDF to inspect small net labels.

6.3 SYS, RCC and clock tree

Under **System Core** → **SYS**, select **Debug** = **Serial Wire** and **Timebase Source** = **TIM6**. Keep the HAL TIM6 handler; reserve the ThreadX port's tick/exception handlers for ThreadX. Avoid duplicate SysTick, PendSV and SVC handlers.

Under **RCC**, select **HSE** = **Crystal/Ceramic Resonator**, not bypass. Keep LSE disabled for this baseline; add RTC/LSE only with a documented timekeeping requirement.

In **Clock Configuration**, use this proposed 96 MHz plan:

Control	Value to set and verify
HSE / PLL source	8 MHz / HSE
PLLM / PLLN	/2, $\times 48 \rightarrow 192$ MHz VCO
PLLR / SYSCLK source	/2, PLLCLK $\rightarrow 96$ MHz
PLLQ / USB mux	/4, PLLQ $\rightarrow 48$ MHz USB

Control	Value to set and verify
AHB / APB1 / APB2	All /1 → 96 MHz
Voltage scale	Range 1; accept valid CubeMX flash latency
SPI1 prescaler	/32 → 3 MHz from PCLK2

Resolve red fields and recheck the whole table if the solver changes anything. Inspect generated clock code for an enabled PLLQ output and the correct USB mux. Seeing a 48 MHz clock elsewhere does not prove USB uses it. No HSI48/CRS path is required for this HSE-derived plan.

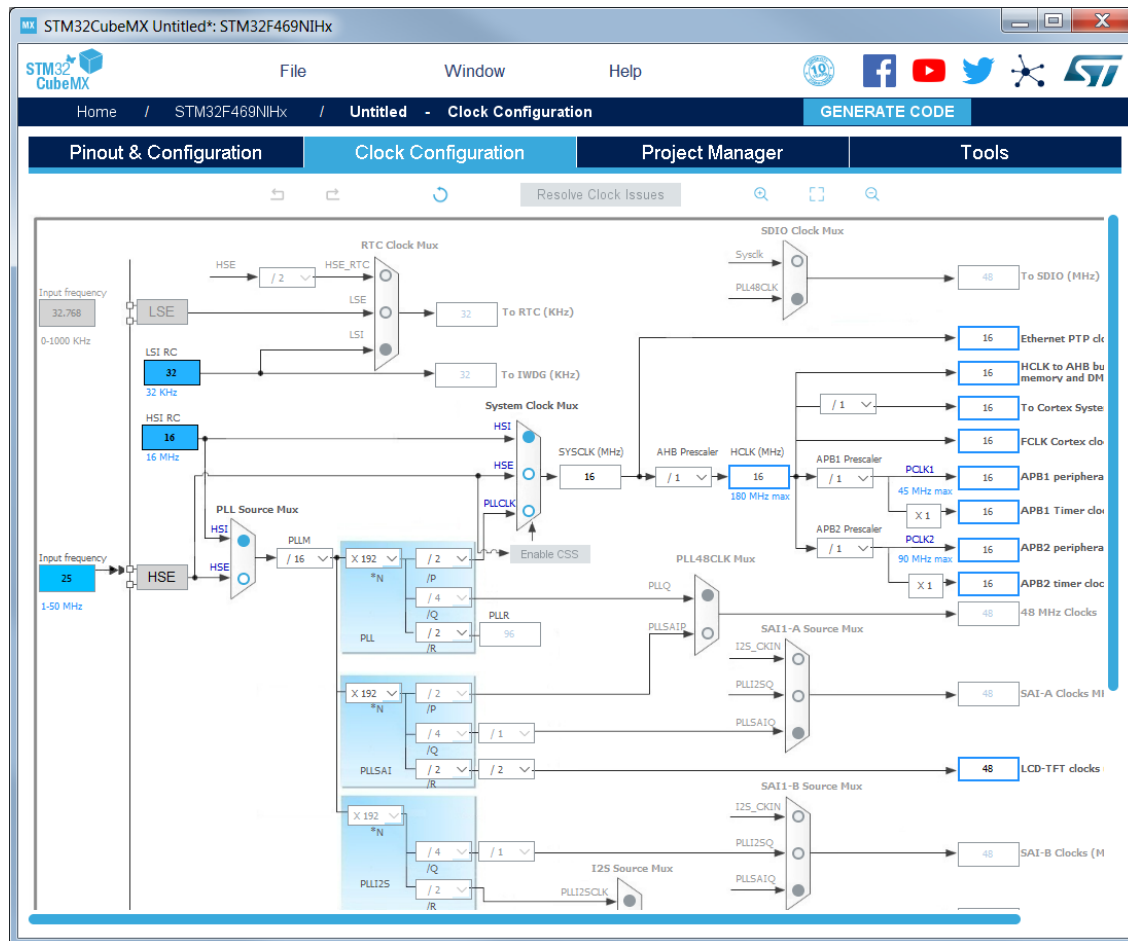


Figure 2: Clock Configuration location: UM1718 figure 183, p.180. This is an F469 example; use the G491 clock values above.

6.4 GPIO and SPI1

Assign the four CS pins to GPIO0_Output. In **System Core** → **GPIO**, set High, push-pull, no pull, Low speed; label them CS_Accel, CS_Gyro, CS_BMP390, CS_Flash. Set PA8/PC13 labels LED/BUZZER, initial Low. A buzzer waveform is not required for logging.

Inspect `MX_GPIO_Init()`: CS output bits must be set high **before** the pins become outputs. Keep all CS high until their transactions. Hardware pull-ups may be needed for deselection during reset, before firmware executes.

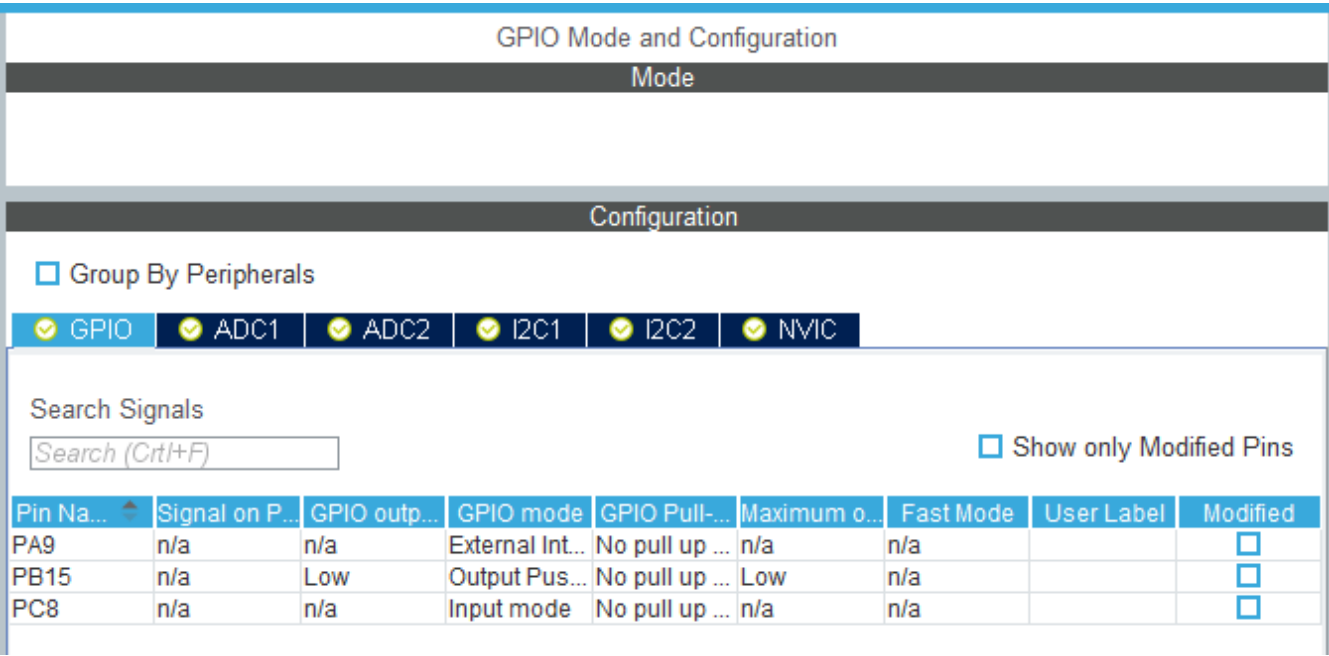


Figure 3: GPIO settings location: UM1718 figure 83, p.107. Replace example rows with the CrashNBurn pins and levels.

Under **Connectivity** → **SPI1**, choose **Full-Duplex Master** and verify PA5/PA6/PA7.

SPI setting	Baseline
Frame / bit order	Motorola / MSB first
Data size	8 bits
CPOL / CPHA	Low / 1 Edge (mode 0)
NSS	Software; pulse disabled
Prescaler	32; displayed rate 3 Mbit/s
CRC / TI mode	Disabled
DMA / SPI interrupt	Disabled for initial bounded polling-HAL driver

Validate common mode-0 timing and per-device dummy bytes against the selected datasheets. Configure sensor ODR/ranges and flash commands in your drivers, not in CubeMX's SPI panel.

6.5 USB device, ThreadX and USBX

Select **Connectivity** → **USB** → **Device (FS)**. Verify PA11 DM and PA12 DP. Use the USB device peripheral, not an OTG host configuration. Do not also enable the legacy `USB_DEVICE` middleware.

Enable the generated device low-priority USB interrupt, normally `USB_LP` for this part. Proposed preemption priorities: **USB 6**, **HAL TIM6 15**, priority grouping 4. Check these against the generated ThreadX Cortex-M4 port and USBX example; this is not a FreeRTOS `MAX_SYSCALL` recipe.

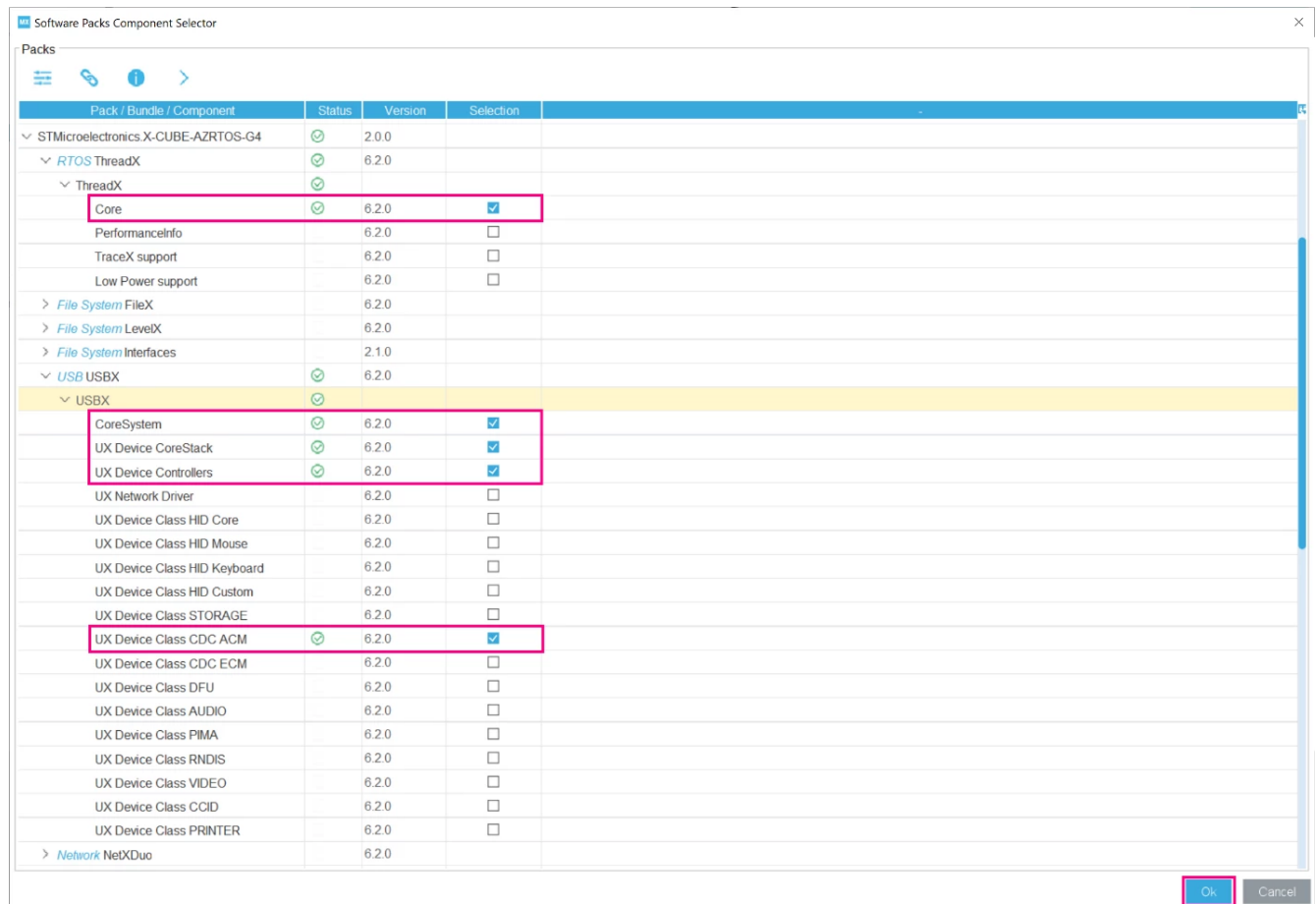


Figure 5: Component selector: ST X-CUBE-AZRTOS-G4 2.0.0 CDC tutorial. The highlighted selections show the components to locate.

Apply, then open the pack configuration. Enable ThreadX/USBX and **register CDC ACM**; including class sources alone may not register the class. Disable unneeded DFU, host classes, FileX, LevelX, low-power and TraceX options for this baseline. Use generated activation/deactivation hooks.

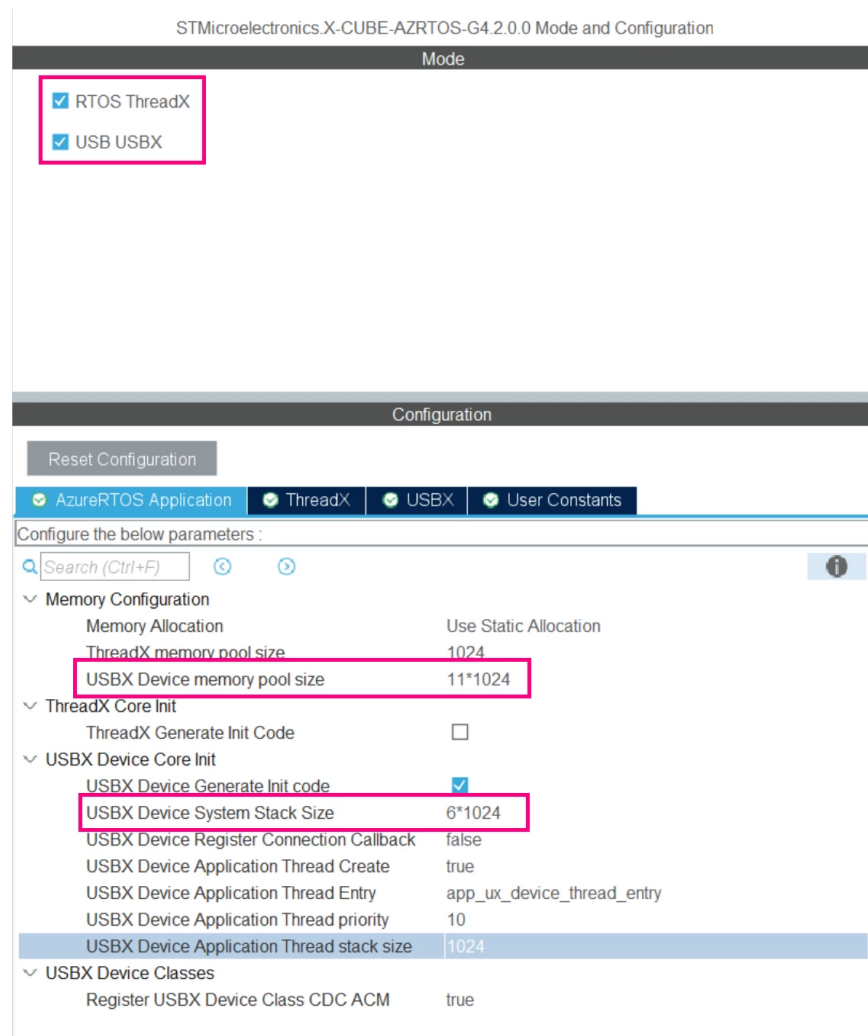


Figure 6: Pack settings: ST tutorial reference. Its example pool sizes and unchecked ThreadX generation box are not the CrashNBurn allocation plan.

Start with static backing storage, a **24 KiB USBX device pool** and **6 KiB device system stack**. Inspect whether the generated stack comes from that pool before counting RAM twice. Allocate the three application stacks and queue arrays separately from the example's small ThreadX pool. Check allocation results and adjust from measured use.

Enable ThreadX init generation when the pack must emit `App_ThreadX_Init()`, or provide the hook yourself, but never define it twice. The pack may generate a sample thread rather than a GUI task list. Create the three workers in the generated user hook after queues/mutex/buffers exist; remove or repurpose the sample worker. Verify `TX_TIMER_TICKS_PER_SECOND` and the actual port tick are both 1 kHz.

Set CDC descriptors and PCD endpoint/packet-memory allocation coherently using the G4 CDC example. Use one control interface and one data interface, a unique serial string, an authorized development VID/PID, and power attributes matching the board. Full-speed bulk packets are at most 64 bytes. Review self-powered VBUS/attach handling from section 2 before enabling the device pull-up.

Keep blocking USB receive/transmit operations in the USB adapter's infrastructure context with bounded buffers. They must not block acquisition or hold the flash/SPI lock. Compare generated integration with [ST's G4 examples](#).

6.6 Generate C and CMake

In **Project Manager** → **Project**, set project name **crashnburn**, verify the destination, select **Toolchain / IDE = CMake**, **Default Compiler / Linker = GCC**, and **Generate under root** where shown. Confirm the final path preview.

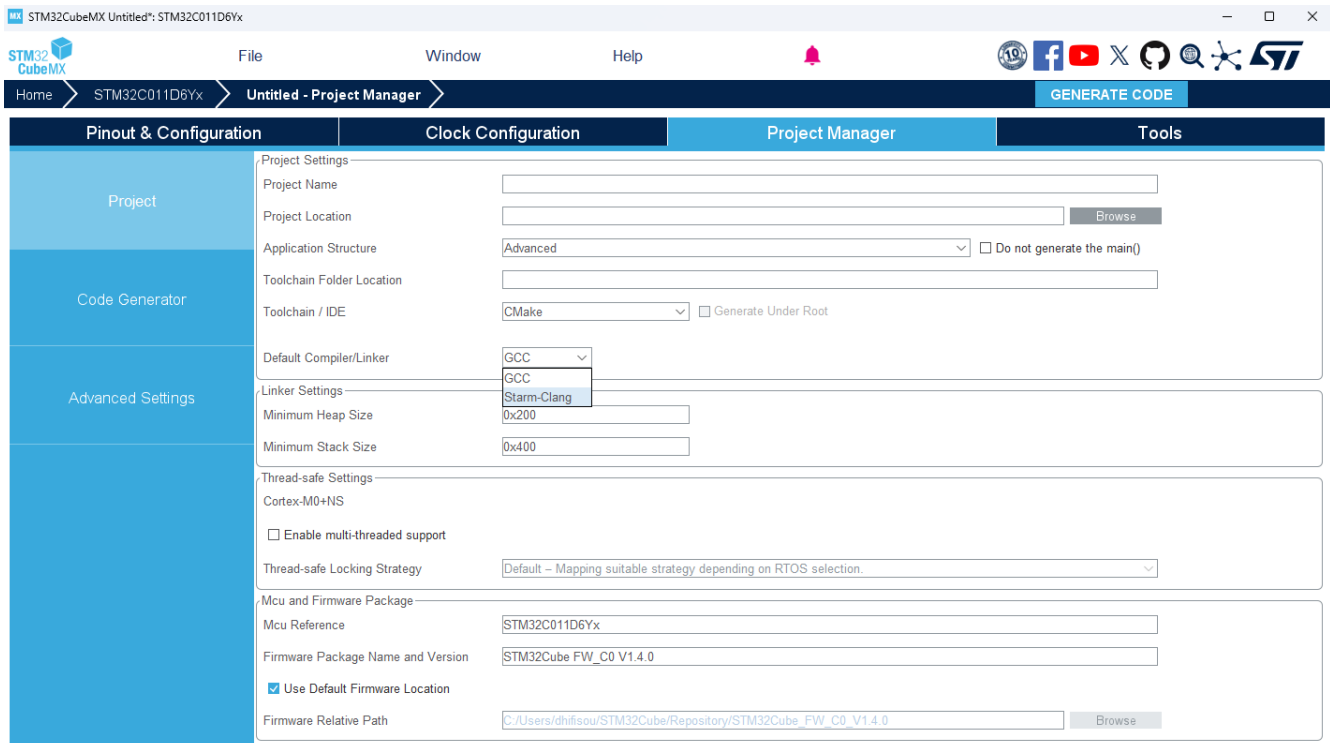


Figure 7: CMake/GCC controls: UM1718 figure 188, p.191. Replace the example name/location with this project's values.

In **Code Generator**, enable **Keep User Code when re-generating** and separate peripheral **.c/.h** files if desired. In **Advanced Settings**, select HAL drivers. Use a portable copied-library layout and record the package version.

Click **GENERATE CODE**, review warnings, and inspect the output:

```
project root/
  build.py
  crashnburn.ioc
  CMakeLists.txt
  CMakePresets.json           # if generated by this version
  cmake/gcc-arm-none-eabi.cmake
  cmake/stm32cubemx/CMakeLists.txt
  Core/Inc/ Core/Src/ Drivers/
  Middlewares/               # ThreadX / USBX
  AZURE_RTOS/ or package application directory
  startup_stm32g491xx.s
  STM32G491...FLASH.ld
```

Paths vary by pack. Ensure CMake includes the ThreadX Cortex-M4 GNU port assembly, USBX device/CDC sources, DCD adapter and application glue. Missing **tx_*** or **ux_*** symbols indicate integration problems. If the pack omits middleware from CMake, add the matching source/include/port definitions in a user-owned CMake file using the supplied GCC example; document the change. Generation alone is not a successful build.

Keep handwritten modules in **App/** or another user-owned directory, with calls inside generated **USER CODE** hooks. A suitable module allocation is sensor/processing/data tasks, shared SPI helper, BMI088/BMP390/flash drivers,

log store and USB command adapter. After creating the files, add them to the actual top-level executable target:

```
target_sources(${CMAKE_PROJECT_NAME} PRIVATE
  App/sensor_task.c App/processing_task.c App/data_task.c
  App/spi_bus.c App/bmi088_driver.c App/bmp390_driver.c
  App/w25q32_driver.c App/log_store.c App/usb_commands.c)
target_include_directories(${CMAKE_PROJECT_NAME} PRIVATE App)
```

Do not paste source entries for files you have not created. Keep custom CMake outside the regenerated subdirectory and check regeneration diffs.

7. Set up VS Code, check dependencies and build

7.1 Open the generated project

Install Visual Studio Code, then **File** → **Open Folder** and select the folder containing `crashnburn.ioc`, `CMakeLists.txt` and `build.py`. Opening only `Core/` hides the build root. Use the **STMicroelectronics STM32CubeIDE for Visual Studio Code extension pack** for this project, including ST's clangd code navigation and STM32 debugging integration.

Follow the ST project-discovery/setup prompt for the generated CMake project. Select the GNU ARM toolchain matching CubeMX's **GCC** output. Do not select a desktop compiler or switch to an Arm Clang compiler just because the editor uses clangd: the language server and firmware compiler are different tools. Review generated/imported presets and confirm the MCU and compiler paths. [ST workspace documentation](#).

Keep the helper's `Debug_Script/Release_Script` build directories separate from any ST-generated editor preset directories. A host-configured CMake cache must not be reused for ARM. The terminal `build.py` workflow below remains the reproducible build baseline.

7.2 Install the STM32 extensions

Open **View** → **Extensions** or select the four-square Activity Bar icon. Search for **STM32CubeIDE for Visual Studio Code**, verify publisher **STMicroelectronics**, and install the complete pack. Its ID is:

```
stmicroelectronics.stm32-vscode-extension
```

Let the pack install and manage its companion extensions. Follow its setup/download prompts and reload the window when requested. [ST installation guide](#).

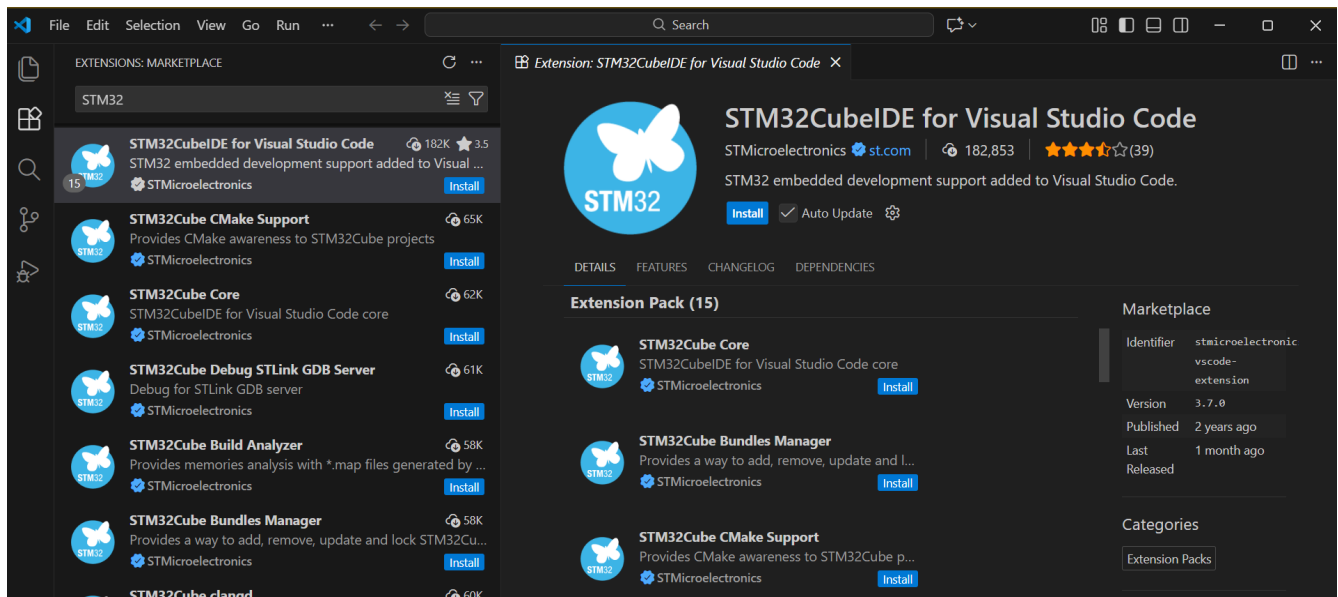


Figure 8: Install the STM32CubeIDE for Visual Studio Code pack from STMicroelectronics. Official ST screenshot shows the pack and companion STM32 extensions; the displayed version is a reference, not a pinned requirement.

The ST setup supplies STM32 project/CMake support, bundle management, **STM32Cube clangd**, probe debugging and related tools. Do not separately install Microsoft's C/C++ extension or the C/C++ Extension Pack for this training workflow. If those are already enabled, select their gear menu and **Disable (Workspace)**, then reload. Use the ST language service so competing C/C++ indexing/diagnostic providers are not active for the same project.

ST may itself include Microsoft CMake Tools as a managed dependency. Leave dependencies required by the ST pack enabled; you do not need to install or configure a separate generic C/C++/CMake extension setup. [ST pack contents](#).

Filter Extensions with `@installed` and verify the ST pack, STM32Cube clangd, CMake/project support and selected ST-LINK debug components are enabled. Use the ST **Bundles Manager** to complete required downloads and select the needed STM32G4 device support. Record the pack and tool versions in the SDD. If upgrading from an older STM32 extension, follow ST's [migration instructions](#).

Other tools, only for the corresponding workflow

- **Git is built in to VS Code**; install the Git executable. No additional Git extension is required for clone, diff, stage, commit, branch or push/pull. [Source Control overview](#).
- **GitHub Pull Requests**, `GitHub.vscode-pull-request-github`, is optional for reviewing a GitHub-hosted project inside VS Code. [Extension](#).
- **Python**, `ms-python.python`, is optional for editing the helper or USB host client. Running the script requires Python itself, not its editor extension. [Extension](#).
- **LaTeX Workshop**, `James-Yu.latex-workshop`, is optional for local SDD editing. Overleaf needs no local LaTeX extension; local compilation still requires XeLaTeX/latexmk. [Extension](#).

To recommend the firmware editor setup to teammates, use this `.vscode/extensions.json`:

```
{
  "recommendations": [
    "stmicroelectronics.stm32-vscode-extension"
  ]
}
```

This recommends the ST pack; it does not install Git, CubeMX, CubeProgrammer, CubeCLT or Python. Complete the dependency checklist in section 7.4 before building.

7.3 Use Git integration in VS Code

Git records local project history. A hosting service such as GitHub stores a remote copy and supports team review. A local **commit** does not upload anything; **push** sends commits to the remote. VS Code supplies the Git UI but calls your installed Git executable.

Check Git and your commit identity

In the integrated terminal, run:

```
git --version
git status
git remote -v
git config --get user.name
git config --get user.email
```

If Git is missing, install it and restart VS Code. If you need a commit identity for this repository, replace the following placeholders with your own details:

```
git config --local user.name "Your Name"
git config --local user.email "your-address@example.org"
```

These settings name the author of commits; they do not sign you in to GitHub. Repository-local configuration avoids changing your other projects. Use your chosen hosting-service no-reply address if that is your established convention. [Git identity configuration](#).

Open or clone the correct repository

If you already have this folder with its `.git` directory, open it directly. Do not initialize another repository inside `Core/` or clone over your working copy. If starting from the team's remote, open the Command Palette and run **Git: Clone**, paste the supplied repository URL, choose a parent folder and open the result. Check `git remote -v` against the team URL. Authenticate through the host's browser/credential flow for HTTPS, or use your configured SSH key for an SSH URL. Git identity and remote authentication are separate. [Repositories and remotes](#).

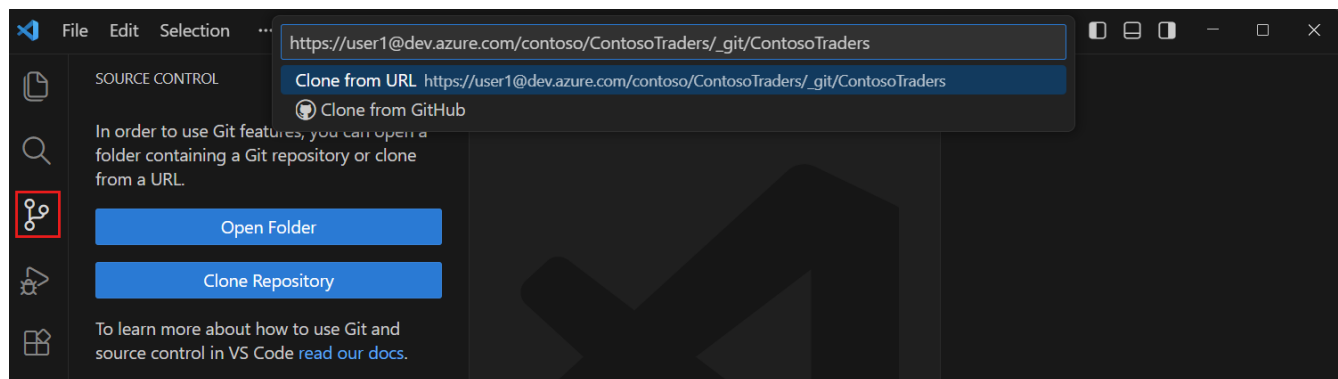


Figure 9: Source Control icon and Clone Repository command. Microsoft reference screenshot shows an example remote; use the team's actual URL and clone only when you need a new working copy.

If starting from an extracted ZIP with no `.git`, use **Initialize Repository** once in the intended project root. Make a baseline commit before creating feature branches. Connect a remote only when the team provides one. An existing repo with no commits also needs that initial baseline; do not delete its `.git` directory.

Make a small, reviewable change

1. With a committed baseline, use **Git: Create Branch** to create a branch such as `sensor-bringup`. Confirm the branch name in the status bar before editing.
2. Make one coherent change—for example, initialize BMP390 and add its identity test notes. Save the files.

3. Open **Source Control**. Select a changed file to inspect the diff. Review CubeMX regeneration carefully, especially custom-code regions and CMake changes.
4. Stage the intended files with **+**. Check **Staged Changes**, enter a descriptive message such as **Configure SPI1 and four chip selects**, then **Commit**. Staging selects the next commit's contents; it does not upload them. [Staging and commits](#).

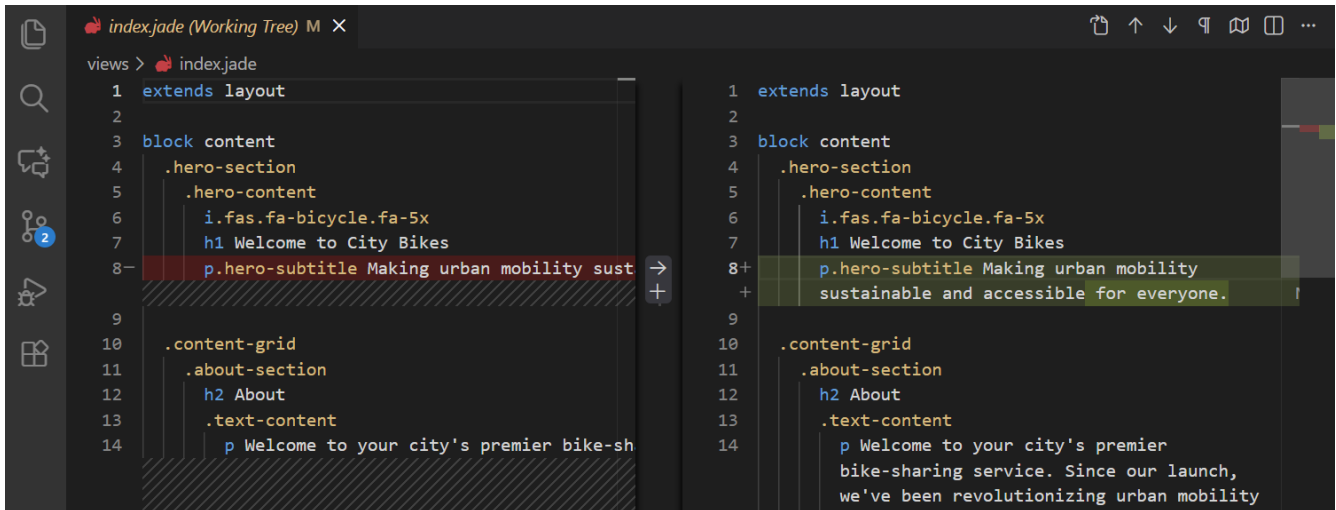


Figure 10: Diff review before staging. Microsoft example compares a web-project file; use this same view for your C, CMake and .ioc changes.

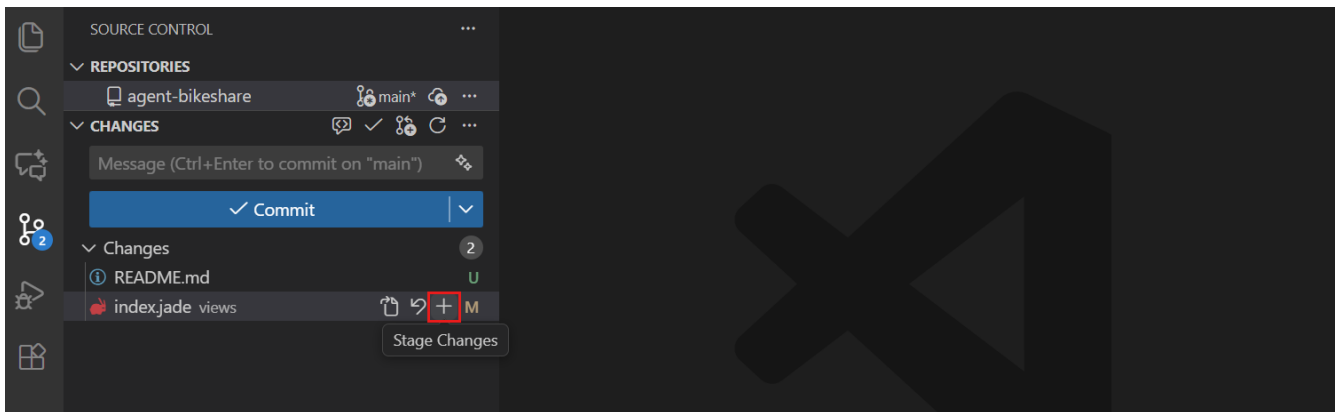


Figure 11: The + button stages a changed file. Microsoft reference screenshot highlights the control; the example repository, files and branch are not CrashNBurn.

For this project, track the `.ioc`, application source/headers, CMake/toolchain files, required startup/linker files and agreed HAL/middleware sources so a teammate can reproduce the build. Include the SDD's editable source/ZIP and exported revision when it changes. Keep `/build/`, Python caches and `.DS_Store` out of normal commits; the included `.gitignore` covers those. Share portable editor recommendations, not machine-specific absolute tool paths. A binary PDF change needs a visual review, because it does not give a useful line-by-line source diff.

Discard Changes removes edits; it is not the same as unstage. If you chose the wrong file for a commit, unstage it and keep the work intact.

Push and request review

Use **Publish Branch** for a new branch, or Source Control's **...** → **Push** for one already tracking a remote. Confirm the remote and branch first. **Fetch** retrieves remote history; **Pull** integrates remote work according to the

repository's Git settings. **Sync Changes** may pull and push, so use explicit commands while learning. [Remote operations](#).

After pushing, create a pull request through the hosting website or the optional GitHub Pull Requests extension. State the behavior changed, design/requirement IDs and real test results. Resolve review feedback with further commits. If a pull produces conflicts, inspect both versions, resolve and rebuild before completing the integration; do not blindly choose “accept all.” Never force-push a shared branch merely to dismiss a rejected push.

Git checkpoint: the correct repository and branch are visible, the intended diff is committed, build output is ignored, and—when collaboration is required—the pushed branch and review request are visible on the team's host. No remote operation is required merely to build locally. Microsoft screenshot sources: [Git quickstart](#); [extension UI](#).

7.4 Dependency checklist: install and verify

Complete this checklist before your first build. You can use it before the CubeMX steps in chapter 6 as well. Install packages for your operating system and CPU architecture, record their versions in the SDD, and verify executables from **VS Code's integrated terminal**.

Required development software

Dependency	Install / purpose	Completion check
Git	Install Git for local history and remote collaboration.	<code>git --version</code> ; Source Control recognizes the repository.
Visual Studio Code	Install VS Code as the editor.	Launch the app; Help/About shows its version. <code>code --version</code> works if its optional shell command is installed.
STM32 VS Code pack	Install the STMicroelectronics pack described in section 7.2.	ST view, clangd, project/build and debug components load without setup errors.
STM32CubeMX	Install CubeMX for MCU configuration and CMake/GCC generation.	Open the app, check Help/About, select STM32G491CET6 and save the .ioc.
STM32CubeG4 firmware	Install in CubeMX's embedded software package manager.	Selected version supplies G491 HAL/CMSIS/startup files.
X-CUBE-AZRTOS-G4	Install the G4 expansion pack .	ThreadX and USBX CDC components are available in CubeMX.
STM32CubeCLT	Install CubeCLT as the baseline command-line toolchain for this training project.	Locate its GNU tools, CMake, Ninja and programmer directories; run the checks below.
CubeProgrammer CLI	Provide STM32CubeProgrammer , standalone or the compatible copy bundled with CLT. The executable is <code>STM32_Programmer_CLI</code> (.exe on Windows).	<code>STM32_Programmer_CLI --version</code> succeeds; record its actual path.
Python 3	Install Python to run <code>build.py</code> .	<code>python3 --version</code> , or <code>py -3 --version</code> on Windows; <code>helper --help</code> succeeds.

CubeCLT versus newer ST bundles: CubeCLT packages command-line tools for external IDE workflows. Newer ST VS Code releases also use their own bundle manager, replacing the older extension's dependency on one monolithic CLT install. Keep **CubeCLT as this project's terminal-build baseline**, while allowing the installed

ST extension to obtain its required components. Do not assume the extension automatically exports its private bundle paths into every terminal. Record which compiler/programmer each workflow uses and align versions where possible. [CubeCLT](#); [ST's tool-bundle transition](#).

If the team intentionally standardizes on ST-managed bundles instead of CubeCLT, record that substitution and expose the same GNU ARM, CMake, Ninja and programmer executables to the helper. The script invokes those executables directly; it does not invoke the ST cube wrapper. Do not install an arbitrary host GCC to satisfy an ARM compiler check.

Command-line tools that must be reachable

Run these commands in **Terminal** → **New Terminal**:

```
git --version
python3 --version
cmake --version
ninja --version
arm-none-eabi-gcc --version
arm-none-eabi-objcopy --version
arm-none-eabi-gdb --version
STM32_Programmer_CLI --version
```

On Windows use `py -3` or `python` if that is your working Python 3 launcher. The helper uses Python's standard library: no `pip install` is required. GDB is needed for GDB-based debugging/flash methods; GCC, objcopy, CMake and Ninja are needed for the helper's normal build. CMake must satisfy the generated project's `cmake_minimum_required(...)`.

A GUI that launches is not proof that its command-line tools are on PATH. Find and record the installed executable locations:

```
# macOS/Linux
command -v git
command -v cmake
command -v ninja
command -v arm-none-eabi-gcc
command -v STM32_Programmer_CLI

# Windows PowerShell
Get-Command git, cmake, ninja, arm-none-eabi-gcc, STM32_Programmer_CLI
```

Add the relevant **bin directories** to your OS/user PATH, then fully restart VS Code. In a typical CubeCLT installation these include `GNU-tools-for-STM32/bin`, `CMake/bin`, `Ninja/bin` and `STM32CubeProgrammer/bin`; inspect your installed release rather than copying a teammate's absolute path. Add the ST-LINK server directory if your selected backend needs it. A standalone CubeProgrammer installation may use a different location.

If the programmer is intentionally not on PATH, verify it by full path and use the helper's `--stm32prog-cli` option. On PowerShell, invoke a quoted executable path with `&`. Installing both standalone CubeProgrammer and CLT is not required when the bundled compatible CLI already passes the check. Pin the chosen path so one build does not silently use a different installed release.

Probe, connection and document dependencies

- **ST-LINK probe and USB data cable:** required for this guide's SWD flash/debug workflow. Confirm actual SWDIO, SWCLK, reference voltage, ground and reset wiring using chapter 2. Power the board through the intended external source.
- **Probe drivers / permissions:** install ST's USB drivers where required, particularly on Windows. On Linux, follow the package's USB/udev access instructions. Use the ST extension's resource links and verify the probe is detected; installing an extension alone does not install every OS driver.
- **Board USB data cable:** required for CDC log export. It is separate from the SWD probe connection and must carry data, not power only. The host downloader must implement the proposed command protocol; this guide does not include that client.

- **Internet access:** needed initially for ST packages/bundles and remote Git authentication; use a browser/SSH credential setup appropriate to the team's Git host.
- **SDD authoring:** use a browser and Overleaf account for the hosted workflow. For local compilation, install a TeX distribution with XeLaTeX and latexmk and check `xelatex --version` and `latexmk -v`. The project ZIPs already contain their logo/style assets.

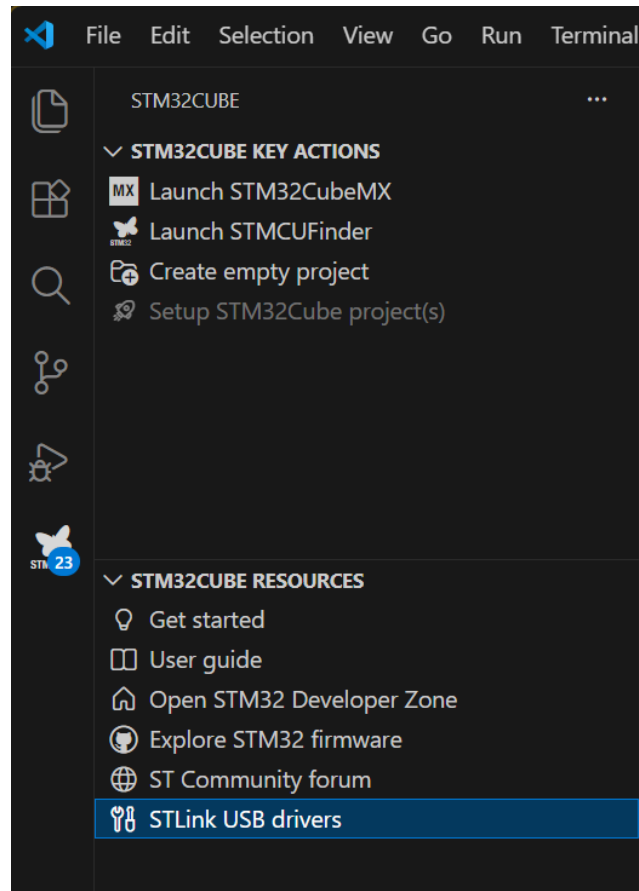


Figure 12: ST's installation documentation shows the STLink USB drivers resource. Follow the driver instructions for your operating system; the screenshot is an ST reference UI example.

Probe driver reference: [ST installation instructions](#).

Finally confirm that CubeMX generated the project files:

```
# macOS/Linux in the project root
ls CMakeLists.txt crashnburn.ioc cmake/gcc-arm-none-eabi.cmake

# Windows PowerShell equivalent
Get-Item CMakeLists.txt, crashnburn.ioc, cmake/gcc-arm-none-eabi.cmake
```

Use the actual generated toolchain path if your CubeMX version differs. Do not create an empty placeholder to pass this check. **Ready-to-build checkpoint:** the ST pack is active, dependencies resolve to recorded versions, the `.ioc/CMake/toolchain` files exist, and `python3 build.py --help` (or the Windows equivalent) succeeds.

7.5 Use build.py

The included helper is adapted from the sibling `analog_tears` repository. It configures CMake/Ninja with an explicit ARM toolchain, builds, and converts ELF to BIN. It only looks for `CMakeLists.txt` beside itself and has no SEDSNet/telemetry option.

```
# macOS/Linux; run from the project root
python3 build.py --help
python3 build.py build --debug
python3 build.py build --release

# Global options precede the subcommand:
python3 build.py --toolchain cmake/gcc-arm-none-eabi.cmake build --debug
python3 build.py --artifact crashnburn build --release

# Fresh cache after an incorrect host-compiler configuration:
python3 build.py --build-subdir Debug_ARM build --debug
```

On Windows, replace `python3` with `py -3` or your working Python 3 launcher. On macOS/Linux, `./build.py` also works when executable.

Outputs normally appear in `build/Debug_Script` and `build/Release_Script`: an `.elf`, a `.bin`, and `compile_commands.json`. Success means configure, compile, link **and** object conversion finished. Read the first actual compiler/linker error when a build fails; the helper's final error is only a summary.

The helper has been exercised with separate minimal ARM Debug and Release projects. No CrashNBurn firmware or hardware test is implied. Until you generate `CMakeLists.txt`, its missing-project message is expected.

7.6 Make ST code navigation follow the ARM build

Use **STM32Cube clangd**, supplied by the ST extension pack, for completion, navigation and diagnostics. Keep the separate Microsoft C/C++ extension disabled in this workspace. Do not create a `c_cpp_properties.json` configuration for this workflow; that belongs to a different language-service setup.

After a successful Debug build, verify the compilation database exists at `build/Debug_Script/compile_commands.json`. ST's project configuration normally sets up code indexing for its selected project/preset. Confirm which build directory it uses; the ST editor preset's database and the helper's database can be in different places. [ST editing documentation](#).

If you use the helper's build as the indexing source and automatic discovery does not find it, merge this setting into the project's `.clangd` file, preserving other ST-generated settings:

```
CompileFlags:
  CompilationDatabase: build/Debug_Script
```

The path is relative to the `.clangd` file. Change it if you chose a different `--build-subdir`. Do not blindly replace an ST-managed configuration: first check whether the extension provides the corresponding active-project/indexing setting, then verify it after reconfiguration. [clangd compilation-database configuration](#).

Open **View** → **Output** and select the STM32Cube clangd/language-server output for your installed version. Check that it loads the intended database and interprets source files with the ARM target/includes from the GNU build. Use the extension's restart/reindex command or reload the window after fixing configuration. Missing standard headers require checking compiler/toolchain discovery, not inventing MCU defines or adding a second language server.

A red editor underline is not necessarily a compiler failure; compare the actual build with the indexing configuration. Keep **GCC** as the firmware compiler for this project even though ST uses **clangd** for the editor.

7.7 Flash, debug and check the board

Resolve the schematic's actual SWDIO wiring, probe reference/ground/reset and external power first. To build then program through CubeProgrammer:

```
python3 build.py flash --debug --method stm32prog-cli \
  --stm32prog-cli /path/to/STM32_Programmer_CLI
```

On PowerShell write the command on one line, using your Python launcher and a quoted Windows executable path. The helper's default address is MCU internal flash 0x08000000. It does not erase/download the external SPI logs.

Its inherited DFU backend requires a compatible DFU bootloader/device; CDC logging is not DFU. Use SWD as the baseline.

For interactive debugging, use ST's extension onboarding with ST-LINK and point the launch configuration at the **Debug ELF you actually built**, such as `build/Debug_Script/crashnburn.elf`. Check the debugger's executable/working directory and tool paths before launching. Editor presets may produce a different ELF; don't debug a stale artifact. Breakpoints pause acquisition and USB service, so timing evidence must come from a free-running test.

Before hardware testing, confirm MCU/startup/linker memory regions match G491CET6; USB is PLLQ 48 MHz; SPI is 8-bit mode 0 at 3 MHz; all CS idle high; TIM6 supplies HAL time; ThreadX owns its exceptions; and USBX is the only device stack. Verify creation/allocation results and inspect RAM/map margins.

Bring up in small checkpoints: LED/clock/SWD → each sensor identity → known physical measurements → scratch-sector flash test → three queues/tasks → durable logs → USB STATUS/LIST → verified export → explicit erase and power-loss recovery. Expected sensor IDs are BMI088 accel `0x1E`, gyro `0x0F`, BMP390 `0x60`; verify against the selected datasheet revisions and check flash JEDEC identity for the fitted variant. Never use a retained-log sector as a scratch test.

Update your SDD with what actually happened: versions, decisions, measured limits, failed tests and fixes. Review every CubeMX regeneration diff, and keep the document's design synchronized with the implementation.

Have fun with the project!